

Arrays and Strings

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

- Declaring and using arrays
- Array initialization
- Operations on arrays
- Fixed- and variable-length arrays
- Arrays and sizeof()
- C Strings
- String functions

Making Arrays

- When you declare an array, you get to specify
 - The **type of its elements**
 - **How many elements** it will have
 - **How** they will be **initialized** (if at all)
- Example:

```
int a[ 10 ];
```
- Notice, part of the type is given **before** the variable name, part **after**.
- In many contexts, the number of elements must be an integer **constant expression**

Meet the Array

- Array notation is similar to Java
- You can access element i in array a : $a[i]$
 - For an n -element array, indices range from 0 to $n-1$
- You can also say it backward: $i[a]$
 - Why can you do this?
 - It's a consequence of what the $a[i]$ notation really means in C
 - We'll learn about this later

C vs Java

- In C, declaring an array variable allocates the array.

```
int a[ 10 ];
```

- In C, arrays can be allocated statically, on the stack or on the heap (later).

- In Java, you need to declare and allocate an array.

```
int[] a = new int [ 10 ];
```

- In Java, all arrays are always allocated on the heap.

Array Initialization

- Initialization rules are like other variables
 - If you don't initialize a **stack-allocated array**, it will contain whatever was previously in that memory.

```
int a[ 20 ];  
for ( int i = 0; i < 20; i++ )  
    printf( "%d ", a[ i ] );
```

outputs

0 1 1423550160 32767 182865920 ...

Yay, garbage!

- If you don't initialize a **statically-allocated array**, it starts out full of zeros.
- You can fill in the elements one at a time.

```
int a[ 20 ];  
a[ 0 ] = 7;  
a[ 1 ] = 14;  
...;
```

Array Initialization

- Or, you can ask the compiler to initialize it for you:

```
int a[ 5 ] = { 7, 14, 21, 28, 35 };
```

- If you initialize just some of the array, the rest will be initialized to zero.

```
int a[ 5 ] = { 7, 14, 21 };
```

Imagine ... , 0, 0

– This is called *partial initialization*.

Array Initialization

- You can let the compiler determine the size
 - by measuring the length of your initialization list.

```
int a[] = { 7, 14, 21, 28, 35 };
```

OK. You must want
a 5-element array.

- C99 offers an alternative initialization syntax

```
int a[] = { [7] = 10, [2] = 40, [5] = 35 };
```

0	0	40	0	0	35	0	10
---	---	----	---	---	----	---	----

Array Initialization

- You can even mix the two notations
 - Where you give an index, C will put the given value there.
 - Where there's no index, C will fill the subsequent index.
 - If you don't give an array size, you'll get an array that's just large enough to hold the last value.

```
int a[] = { 1, [7] = 10, 90, [2] = 40, 2,  
           [5] = 35 };
```

1	0	40	2	0	35	0	10	90
---	---	----	---	---	----	---	----	----

Array Bounds Checking

- C does **no bounds checking** for array access
 - Like many things in C, this decision favors performance over security
 - And, in many situations, the compiler may not even know the size of the array anyway.

```
int a[ 6 ] = { };
```

more program memory ...

0 0 0 0 0 0

... more program memory

- What do you get if you index off the end?
 - Let's find out.

Looking at Memory

- Example program howdyNeighbor.c

```
int x = 25;
int a[ 3 ] = { 10, 20, 30 };
int y = 50;

printf( "Array a: %d %d %d\n", a[ 0 ], a[ 1 ], a[ 2 ] );

printf( "A little before a: %d %d\n", a[ -2 ], a[ -1 ] );

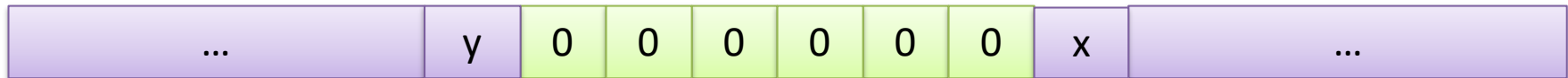
printf( "A little after a: %d %d\n", a[ 3 ], a[ 4 ] );
```

- Output:

```
Array a: 10 20 30
A little before a: 4195712 50
A little after a: 25 1876150896
```

Looking at Memory

- From the example, I think this is what we're seeing:



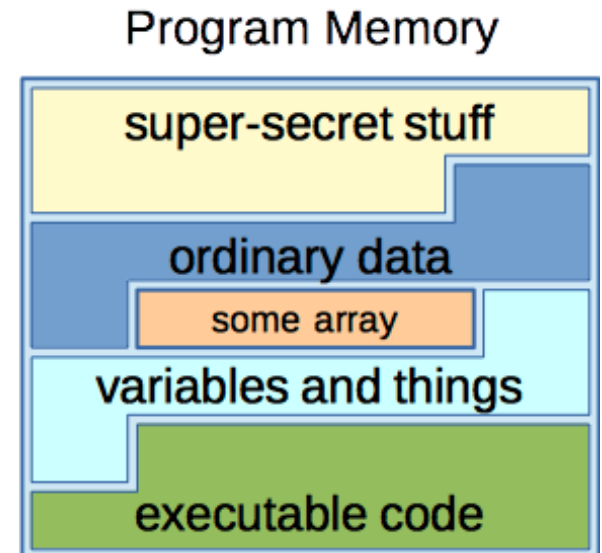
- In this example, we're looking around in other parts of the *stack frame*.
 - Where the function stores its local variables and other values it needs.
- So every array is like a little window into our program's memory

Array Bounds Checking

- Accessing memory outside an array's bounds can do lots of neat things.
 - Accidentally overwrite some of your program's variables
 - Segmentation fault, bus error.
- These errors can be difficult to debug.
 - Maybe you overwrite an important variable or something your program needs to keep running.
 - If you're lucky, this may crash your program right away.
 - If you're unlucky, your program may crash ... later.

With Great Power ...

- In Java, a bad index has defined behavior ... it will throw an exception.
- C says behavior is **undefined** ... it could do anything ... probably nothing good ... for us.
- Accidentally writing past array bounds could be an exploitable vulnerability in our programs.
- An attacker could use this to change the data or behavior of a program.



Buffer Overflow

- A *buffer overflow* happens when a program writes into memory past the end of an array.
 - Typically, while reading input from a user or another program.
- It's an easy mistake to make:

```
printf( "Please enter some numbers\n" );  
  
int list[ 20 ];  
int i = 0;  
while ( scanf( "%d", &list[ i ] ) == 1 )  
    i++;
```

Operations on Arrays

- Normally, we work with an array one element at a time.
- C syntax includes very little for working with the whole array at once.
 - You **can't** copy from one array to another:

```
list2 = list;
```

- Or concatenate two arrays, etc.

```
list3 = list + list2;
```

Operations on Arrays

- Instead, we typically need **loops** to process everything in an array:

```
int list2[ SIZE ];  
for ( int j = 0; j < SIZE; j++ ) {  
    list2[ j ] = list[ j ];  
}
```

```
int list3[ SIZE + SIZE ];  
for ( int j = 0; j < SIZE; j++ )  
    list3[ j ] = list[ j ];  
for ( int j = 0; j < SIZE; j++ )  
    list3[ SIZE + j ] = list[ j ];
```

Choosing Array Size

- The size of a statically allocated array must be a constant expression.

```
int sequence[ 100 ];  
  
int main()  
{  
    ...
```

- This is true in some other contexts also.
- ... and used to be true for stack-allocated arrays.

Variable-Length Arrays

- C99 permits the size of stack-allocated arrays to be determined during execution.

```
int main( void )
{
    int n;

    printf( "How many values would you like: " );
    scanf( "%d", &n );

    int list[ n ];

    ...
}
```

This is called a
variable-length array.

What if n is zero?

or negative?

Variable-Length Arrays

- Variable-length arrays aren't resizable
 - Once you choose the size, you're stuck with it for the array's lifetime.

```
int n;  
printf( "How many values? " );  
scanf( "%d", &n );  
int list[ n ];
```

... do some stuff...

```
printf( "Now how many? " );  
scanf( "%d", &n );  
int list[ n ];
```

This won't work; looks like you're trying to declare a new array.

Variable-Length Arrays

- But, you can choose a different size every time the variable goes into scope:

```
for ( int i = 0; i < 10; i++ ) {  
    int n;  
    printf( "How many values? " );  
    scanf( "%d", &n );  
    int list[ n ];  
  
    ...;  
}
```

This will work.
A new array on each
iteration.

Variable-Length Array Initialization

- Initialization syntax doesn't work with variable-length arrays. ☹️
 - Maybe that's no surprise. How many values would you need?

```
int a[ n ] = { 1, 2, 3 };
```

- Not even the empty initializer syntax. ☹️

```
int a[ n ] = { };
```

Array Size in Memory

- You remember the `sizeof` operator ... right?
- It reports the size of its argument, including array arguments.
 - An array's size is the number of elements times the size of each element.

```
#include <stdlib.h>
```

```
int a;
```

```
float b[100];
```

```
double c[ 16 ];
```

```
size_t aSize = sizeof( a );
```

```
size_t bSize = sizeof( b );
```

```
size_t cSize = sizeof( c );
```

Three variables

How much memory does each one take?

Computing Array Size

- The sizeof operator can (sometimes) let you figure out array length (number of elements).

```
float b[ 100 ];
```

```
...
```

```
int len;
```

```
len = sizeof( b ) / sizeof( b[0] );
```

It may not look like it, but this is really a constant expression.

Why not just say 100 instead?

This is a great trick ... but it **doesn't work everywhere.**

- Here, sizeof() can be evaluated at compile time.

Arrays and Stack Space

- Arrays can be stack-allocated.
- But, in practice, stack space is limited.
- A big array on the stack is an easy way to use it up.

I'm probably going to segfault.

- Or, deeply nested recursion will do it too.

```
int bill()  
{  
    int smallTable[ 10 ];  
  
    smallTable[ 0 ] = 1;  
    return smallTable[ 0 ];  
}
```

```
int bill()  
{  
    int bigTable[ 100000000 ];  
  
    bigTable[ 0 ] = 1;  
    return bigTable[ 0 ];  
}
```

Allocating Large Arrays

- Larger arrays can be allocated statically
 - The compiler can plan ahead for static storage.

```
int bigTable[ 100000000 ];

int bill()
{
    bigTable[ 0 ] = 1;
    return bigTable[ 0 ];
}
```

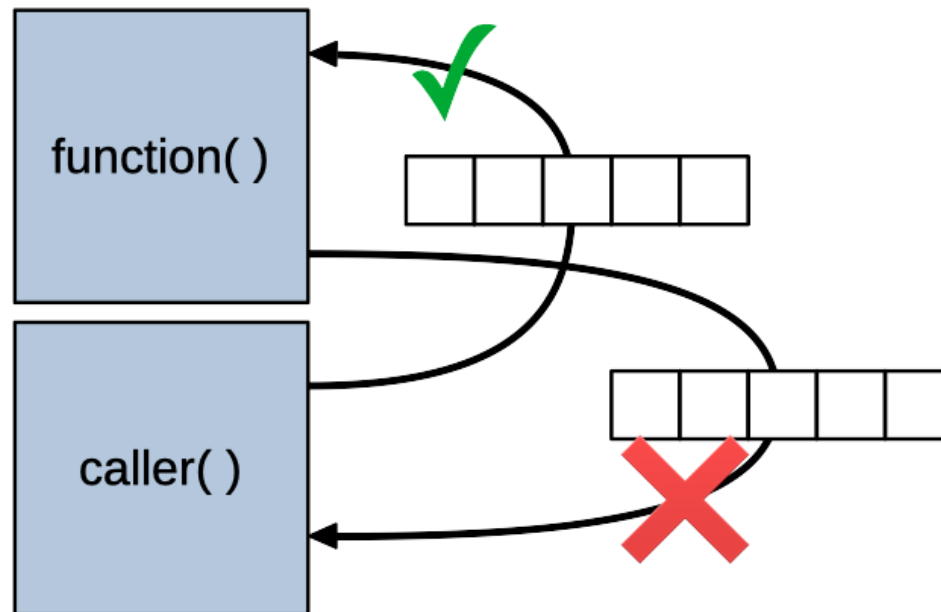
```
int bill()
{
    static int bigTable[ 100000000 ];

    bigTable[ 0 ] = 1;
    return bigTable[ 0 ];
}
```

- Or a large array could be dynamically allocated
... more about this later.

Arrays and Functions

- Functions can have array parameters
- ... but not array return values.



Arrays as Parameters

- Arrays are automatically **passed by reference**

```
void printList( int list[5] )  
{  
    for ( int i = 0; i < 5; i++ )  
        ...;  
}
```

```
int a[5] = { 7, 20, 35, 51, 77 };  
...  
printList( a );
```

a

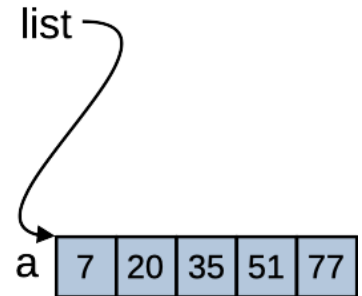
7	20	35	51	77
---	----	----	----	----

Arrays as Parameters

- Arrays are automatically **passed by reference**
 - The function uses the same array used by the caller (not a copy of it)

```
void printList( int list[5] )  
{  
    for ( int i = 0; i < 5; i++ )  
        ...;  
}
```

```
int a[5] = { 7, 20, 35, 51, 77 };  
...  
printList( a );
```

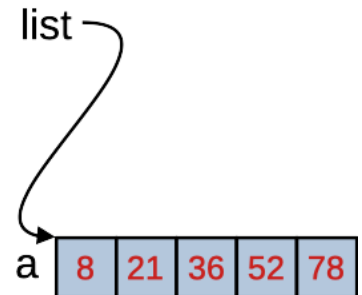


Arrays as Parameters

- Arrays are automatically **passed by reference**
 - The function can change the contents of the array
 - ... and the changes will still be there after the function returns (just like in Java).

```
void addList( int list[5] )  
{  
    for ( int i = 0; i < 5; i++ )  
        list[ i ] += 1;  
}
```

```
int a[5] = { 7, 20, 35, 51, 77 };  
...  
addList( a );
```



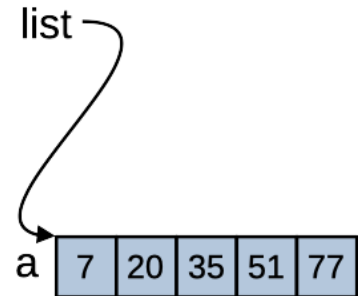
Array Parameter Size

- An array parameter can omit the array size (this is common)
 - The function can be called on an array of any size.

```
void printList( int list[] )  
{  
    for ( int i = 0; i < 5; i++ )  
        ...;  
}
```

No size needed here.

```
int a[5] = { 7, 20, 35, 51, 77 };  
...  
printList( a );
```



Array Parameter Size

- If an array parameter could have any number of elements ...
- ... how can the function tell how many there are?

```
void printList( int list[] )  
{  
    for ( int i = 0; i < 5; i++ )  
        ...;  
}
```

How many elements
are there?

```
int a[7] = { 7, 20, 35, 51  
            77, 84, 103 };  
printList( a );
```

Array Parameter Size

- Using `sizeof()` won't work here.
 - It will compile, but it doesn't give you the value you want.

```
void printList( int list[] )  
{  
    int len = sizeof( list ) / sizeof( int );  
    for ( int i = 0; i < len; i++ )  
        ...;  
}
```

This doesn't work here!

```
int a[8] = { 7, 20, 35, 51  
            77, 84, 103, 118 };  
printList( a );
```

Array Parameter Size

- Using sizeof() won't work
 - It will compile, but it does not do what you want.

**This
doesn't
work here!**

```
void printList( int list )  
{  
    int len = sizeof( list ) / sizeof( int );  
    for ( int i = 0; i < len; i++ )  
        ...;  
}
```

```
int a[8] = { 7, 20, 35, 51  
            77, 84, 103, 118 };  
printList( a );
```

Array Parameter Size

- Typically, you need an extra parameter to pass the array length.

Like this.

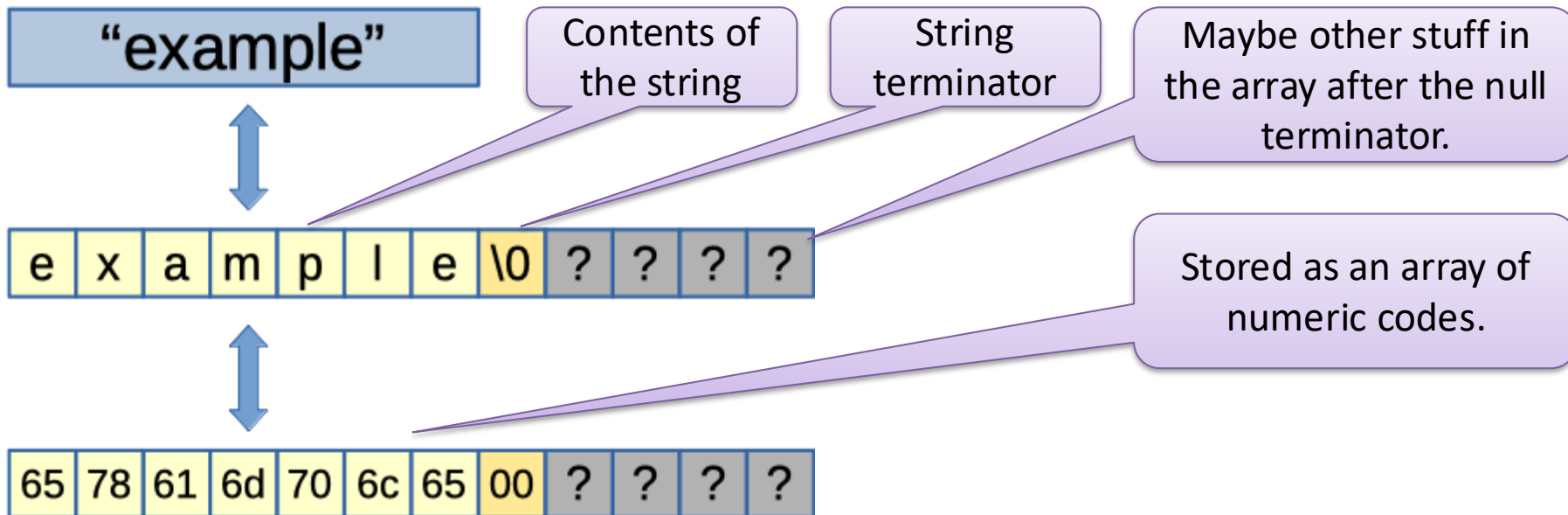
```
void printList( int len, int list[len] )
{
    for ( int i = 0; i < len; i++ )
        ...;
}
```

len isn't needed here,
but useful for
documentation.

```
int a[9] = { 7, 20, 35, 51, 77
            84, 103, 118, 142 };
printList( 9, a );
```

Working with Strings

- A string is a sequence of character codes
 - Just like in java
- In C, a string is stored as a char array.
 - With a zero byte to mark the end of the string.



Initializing a String

- Strings have a short-hand initialization syntax.
 - Automatically puts a null terminator at the end.
- ```
char astr[10] = "hello";
```
- This is another example of partial initialization.
- ... remaining elements will be set to zero.
- You can let the compiler determine array size.
  - Reserves exactly enough space for the **contents** and the **terminator**.

```
char astr[] = "hello";
```

# Strings in C

- To store a string, you can make a char array.
  - Be sure to reserve enough space for ...
  - ... the string's contents ...
  - ... plus one more element for the null terminator.

```
char astr[10];
char bstr[25];
```

I can store a 9-character string.

I can store a 24-character string.

# Iterating over a String

- The null terminator marks the end of the string's body.
- The null terminator evaluates to false, every other character evaluates to true.
- This is how you'd normally iterate over characters in a string:

```
for (int i = 0; str[i]; i++)
 if (str[i] >= 'a' &&
 str[i] <= 'z')
 str[i] = '#';
```

# String Parameters

- Normally, we might need to pass array length to a function.

```
void processList(int len, int list[len]);
```

- But not with strings.
  - The null terminator says how long the string is
  - ... without needing to pass this as an extra parameter.

```
void processString(char str[]);
```

# String I/O

- `printf()` can print the string in a char array
- That's what the conversion specifier `%s` does.

```
char name[] = "Theodore";
printf("Hello %s\n", name);
```

# String I/O

- `scanf()` will read strings with `%s`
  - It reads a single word
  - A space-delimited sequence of non-whitespace characters.
- `scanf` follows the convention of null-terminating the string it reads.

I have room for a 29-character string.

```
char str[30];
```

```
scanf("%s", str);
```

Read a word and put it in this string.

# Reading Strings

```
char str[30];
scanf("%s", str);
```

No & on a string parameter..

- Why no & on the str parameter?
  - Arrays are automatically passed by reference.
  - We don't need extra syntax to ask for this.

```
char str[30];
scanf("%s", str);
```

Look, a buffer-overflow vulnerability.

# Avoiding Buffer Overflow

- scanf() lets you limit the length of a word you will read.
  - That's what the field width

```
char str[30];
scanf("%29s", str);
```

I have room for a 29-character string.

I won't store more than 29 characters.

Then the null terminator.

# Copying Strings

- You can't copy between strings with the assignment operator.

```
char buffer[100] = "abc123";
buffer = "Welcome to C";
```

This works, at initialization time, but not for assignments in general.

- This doesn't work for arrays ...  
and strings are stored in arrays ...  
so 😞
- ... but, there are library functions to help us out  
😊

# Help from the Standard Library

- The C standard library offers functions to work with strings.
- String functions are declared in the `string.h` header:

```
#include <string.h>
```

- All these functions expect `null terminated strings`.
- Today, we'll look at functions to:
  - Measure string length
  - Copy between strings
  - Compare strings
  - Convert strings to ints, doubles, longs, etc.

# String Length

- We have `strlen()` to obtain string length:

```
size_t strlen(const char s[]);
```

- Counts characters up to (but not including) the null terminator.

- The prototype tells us a lot about the function

I take an array of characters.

But, I promise not to modify it.

```
size_t strlen(const char s[]);
```

I return the length, as a `size_t` (which will convert to `int` if you want)

# String Length

- It's easy to use:

```
int len = strlen(name);
```

- It's not too smart.
  - It just looks for the null terminator.
  - So, it has a cost that's linear in the string length.
- This is a bad way to iterate over a string:

```
for (int i = 0; i < strlen(word); i++)
 ...;
```

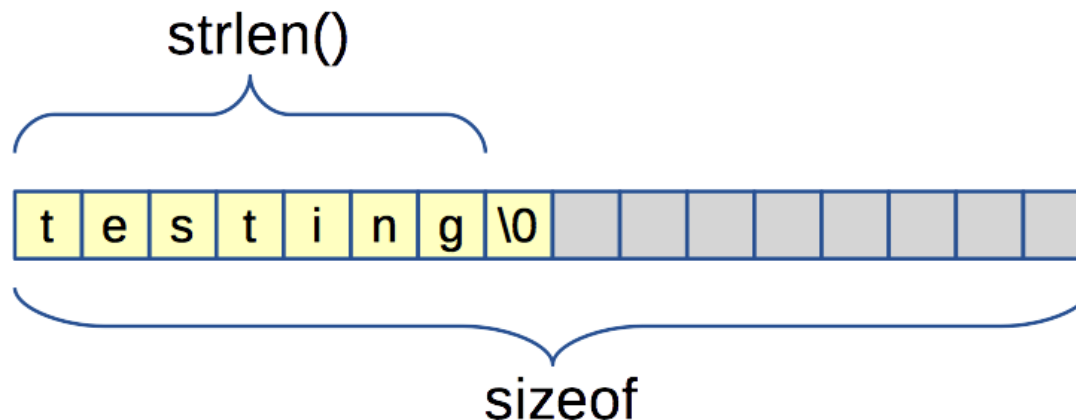
- This is much more efficient (and maybe easier)

```
for (int i = 0; word[i]; i++)
 ...;
```

# strlen() vs sizeof

- Sometimes, people have some confusion between strlen() and sizeof.

```
char buffer[16] = "testing";
```



# strlen() vs sizeof

```
char buffer[16] = "testing";
```

I'm an operator, usually evaluated at compile time.

```
size_t s = sizeof buffer;
```

I don't depend on what's in the string. I'll say 16.

```
size_t n = strlen(buffer);
```

I'm a function call, executed at runtime.

```
buffer[1] = '\\0';
n = strlen(buffer);
s = sizeof buffer;
```

I'll say 7. I depend on what's in the string.

Now, I'll say 1.

But still 16 here.

# Copying Strings

- We have strcpy() to copy strings:

```
void strcpy(char dest[], char src[]);
```

Return type  
isn't really void

- Copies from src to dest, up to and including the null terminator.
- Return a pointer to the start of the destination string (in case you want to do something else with it)

Before

|   |   |   |   |   |   |    |  |  |  |
|---|---|---|---|---|---|----|--|--|--|
| b | a | n | a | n | a | \0 |  |  |  |
|---|---|---|---|---|---|----|--|--|--|

|   |   |   |   |    |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|
| b | o | o | k | \0 |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|

After

|   |   |   |   |    |   |    |  |  |  |
|---|---|---|---|----|---|----|--|--|--|
| b | o | o | k | \0 | a | \0 |  |  |  |
|---|---|---|---|----|---|----|--|--|--|



|   |   |   |   |    |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|
| b | o | o | k | \0 |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|

# Copying Strings

- We can copy from literal strings

```
strcpy(myString, "Greetings Program");
```

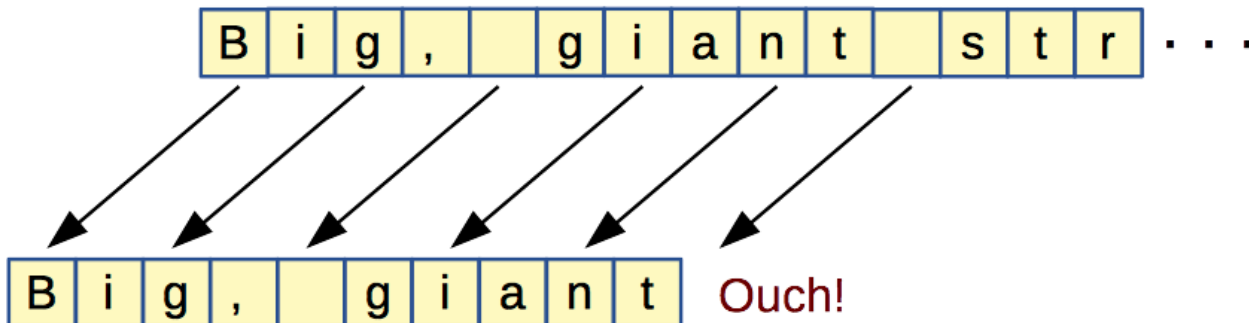
- We can copy from one char array to another.

```
strcpy(myString, name);
```

# Copying Strings

- But, there's a potential for buffer overflow:

```
strcpy(myString, "Big, giant string that you "
"weren't planning for and you don't have enough "
"room to store");
```



# Copying Strings, with Restraint

- Its friend, `strncpy()`, can make it easier to prevent this.

```
void strncpy(char dest[], char src[], size_t n);
```

- It takes an extra parameter, `n`, limiting the number of characters written to `dest`.
- It pads with zeros up to `n`

```
strncpy(dest, src, 10);
```

Before

|   |   |   |   |   |   |    |  |  |  |
|---|---|---|---|---|---|----|--|--|--|
| b | a | n | a | n | a | \0 |  |  |  |
|---|---|---|---|---|---|----|--|--|--|

|   |   |   |   |    |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|
| b | o | o | k | \0 |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|

After

|   |   |   |   |    |    |    |    |    |    |
|---|---|---|---|----|----|----|----|----|----|
| b | o | o | k | \0 | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|----|----|----|----|----|----|



|   |   |   |   |    |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|
| b | o | o | k | \0 |  |  |  |  |  |
|---|---|---|---|----|--|--|--|--|--|

# Copying Strings, with Restraint

- So, this might be typical usage, if it did the right thing ... which it doesn't.

```
strncpy(dest, src, sizeof(dest));
```

- **Caution**, strncpy() won't null terminate if it runs out of room.

Before

|   |   |   |   |   |   |    |  |  |  |
|---|---|---|---|---|---|----|--|--|--|
| b | a | n | a | n | a | \0 |  |  |  |
|---|---|---|---|---|---|----|--|--|--|

|   |   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|---|----|
| b | i | b | l | i | o | m | a | n | i | a | \0 |
|---|---|---|---|---|---|---|---|---|---|---|----|

After

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| b | i | b | l | i | o | m | a | n | i |
|---|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|---|----|
| b | i | b | l | i | o | m | a | n | i | a | \0 |
|---|---|---|---|---|---|---|---|---|---|---|----|

# Copying Strings, with Restraint

- If we want to guarantee null termination, we will need to do it ourselves:

```
strncpy(buffer, word, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';
```

Before

|   |   |   |   |   |   |    |  |  |  |
|---|---|---|---|---|---|----|--|--|--|
| b | a | n | a | n | a | \0 |  |  |  |
|---|---|---|---|---|---|----|--|--|--|

|   |   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|---|----|
| b | i | b | l | i | o | m | a | n | i | a | \0 |
|---|---|---|---|---|---|---|---|---|---|---|----|

After strncpy()

|   |   |   |   |   |   |   |   |   |  |  |  |
|---|---|---|---|---|---|---|---|---|--|--|--|
| b | i | b | l | i | o | m | a | n |  |  |  |
|---|---|---|---|---|---|---|---|---|--|--|--|



|   |   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|---|----|
| b | i | b | l | i | o | m | a | n | i | a | \0 |
|---|---|---|---|---|---|---|---|---|---|---|----|

After null termination

|   |   |   |   |   |   |   |   |   |    |  |  |
|---|---|---|---|---|---|---|---|---|----|--|--|
| b | i | b | l | i | o | m | a | n | \0 |  |  |
|---|---|---|---|---|---|---|---|---|----|--|--|

If we already had a null terminator,  
no harm done, but if we didn't ...

# Comparing Strings

- We have strcmp() to compare strings:

```
int strcmp(char s1[], char s2[]);
```

- *Lexicographically* compares strings s1 and s2, returning:
  - Less than zero if s1 is before s2
  - Zero if they are equal
  - Greater than zero if s1 is after s2
- Stops when it hits a null terminator or a character on which they differ.

|   |   |   |   |   |   |    |  |  |  |
|---|---|---|---|---|---|----|--|--|--|
| b | a | n | a | n | a | \0 |  |  |  |
|---|---|---|---|---|---|----|--|--|--|

↑

|   |   |   |   |   |    |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|
| b | a | n | j | o | \0 |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|

↑

# Comparing Strings

- We can use it to compare strings for equality:

```
if (strcmp(name1, name2) == 0)
 ...;
```

- It's easy to make a mistake if you just use strcmp() as a truth value.

```
if (strcmp(passwd, "joshua"))
 printf("They're equal\n");
```

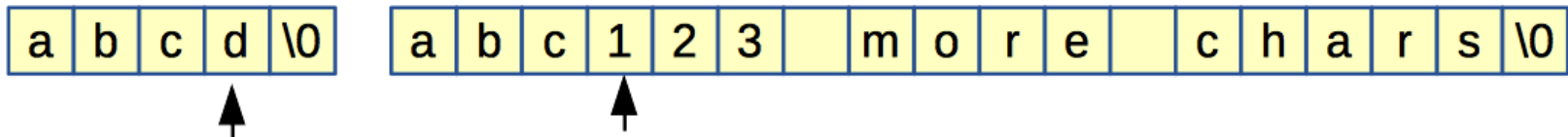
Oops. I guess they're not equal.

# Comparing With Caution

- There are bounded versions of lots of the string-handling functions.
- Want to look at just part of a string? We've got a function for that:

```
int strncmp(char s1[], char s2[],
 size_t n);
```

- Compares at most n characters in s1 and s2.
- Useful if one of your strings isn't null terminated
- ... or you just want to look at the start of a string.



# Literal String Concatenation

- The compiler will automatically concatenate consecutive string literals.

```
char s1[] = "Now " "is " "the " "time";
int len = strlen(s1);
```

I'm 15 characters

- This is a way to break up really long string literals (e.g., over multiple lines).
- It doesn't work for arbitrary strings.

```
char s3[] = s1 s1;
```

This won't work.

# Concatenating Strings

- There's a standard library function to do concatenation.

```
void strcat(char dest[], char src[]);
```

- Copies the string from source to the end of dest.
- Overwriting the null terminator and writing out a new one.

Before

|   |   |   |   |   |    |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|
| a | p | p | l | e | \0 |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|

|   |   |   |    |  |  |  |  |  |  |
|---|---|---|----|--|--|--|--|--|--|
| p | i | e | \0 |  |  |  |  |  |  |
|---|---|---|----|--|--|--|--|--|--|

After

|   |   |   |   |   |   |   |   |    |  |
|---|---|---|---|---|---|---|---|----|--|
| a | p | p | l | e | p | i | e | \0 |  |
|---|---|---|---|---|---|---|---|----|--|



- You have to consider possible buffer overflows.

Before

|   |   |   |   |   |    |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|
| a | p | p | l | e | \0 |  |  |  |  |
|---|---|---|---|---|----|--|--|--|--|

|   |   |   |   |   |   |   |    |  |  |
|---|---|---|---|---|---|---|----|--|--|
| o | r | c | h | a | r | d | \0 |  |  |
|---|---|---|---|---|---|---|----|--|--|

After

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| a | p | p | l | e | o | r | c | h | a |
|---|---|---|---|---|---|---|---|---|---|



Ouch!

# Concatenating Strings

- `strncat()` can help.
  - It lets you give a bound on the length of the string appended.

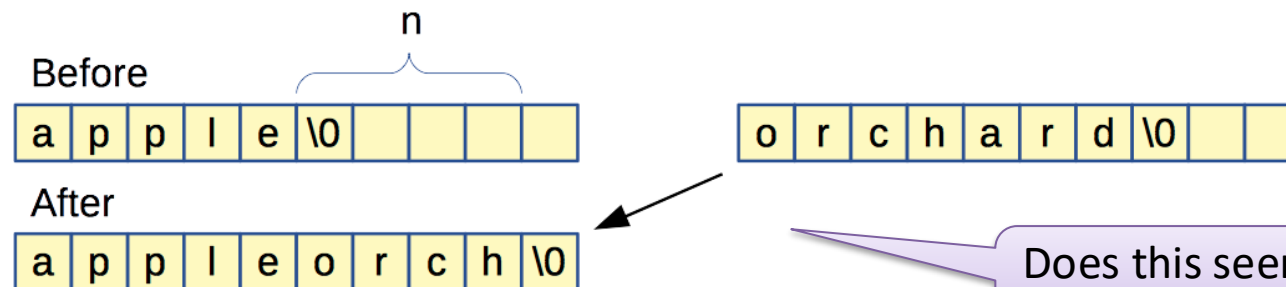
```
void strncat(char dest[], char src[], size_t n);
```

- So, you might use it like:

```
strncat(dest, src, 4);
```

- Or, more generally:

```
strncat(dest, src, sizeof(dest) - strlen(dest) - 1);
```



Does this seem inconsistent with `strncpy()`? It does to me.

# Parsing Numeric Values

- We can parse strings as numeric values
- Say you want to parse an int out of a string?
  - atoi() will do that for you (declared in **stdlib.h**)

```
int atoi(char str[]);
```

String goes in here.

int comes out here.

# Parsing Numeric Values

- `atoi()` has some friends, for other types of conversions

- There's `atof()` for parsing strings as doubles.

```
double atof(char str[]);
```

- There's `atol()` for parsing strings as long ints.

```
long atol(char str[]);
```

- There's `atoll()` for parsing strings as long long ints.

```
long long atoll(char str[]);
```

- These functions are OK (but I almost never use them)

# Problems with atoi() and Friends

- Given an invalid parameter value, these functions return zero.
- So, you can't tell the difference between:

```
int x = atoi("0");
```

and

```
int x = atoi("garbage");
```

- Later, we'll see a general-purpose way to do all these conversions.